Lecture 0
Roadmap: What Git Is, Where It Came From, and How This Course Works

Version control, a short history, Git versus GitHub, and the order of the lectures

Learning goals

- Explain the problem that version control solves, in your own words.
- Tell apart local, centralised and distributed version control.
- Know the key moments in Git's history and a few quick facts about it.
- Explain the difference between *Git* and *GitHub*.
- Know the order of the lectures and how homework, checkers and solutions work.

1 The problem: files change, and we forget how

Have you ever had a folder like this?

```
report.docx
report_v2.docx
report_v2_final.docx
report_v2_final_REALLY_final.docx
report_v2_final_REALLY_final_use_this_one.docx
```

It works for a while, then it breaks down. Which file is the newest? What exactly changed between `v2` and `final`? What if a friend edits one copy while you edit another? Whose changes win? And if you delete a paragraph on Monday and need it back on Friday, where is it?

A **version control system** (VCS) solves all of these. It is a tool that

- records the history of your project as a series of saved states (*versions*);
- lets you compare any two versions and go back to an old one;
- lets several people work at the same time and combine their work.

You keep *one* folder, and the tool remembers every earlier state of it.

Key idea

Version control is a time machine plus a collaboration tool for your files. It is used for code, but also for papers, notes, thesis chapters and configuration files: anything made of text.

Quick fact

Words you will meet soon (each is explained properly later: *remote* in Lecture 6, the others in Lectures 1–5). A *repository* is a project folder together with its saved history. A *commit* is one saved version. A *branch* is a separate line of work. A *remote* is a copy of the repository stored somewhere else, such as GitHub. Do not worry if these feel vague now.

2 Three generations of version control

Version control tools came in three broad kinds. The diagram shows where the history of the project is stored in each.

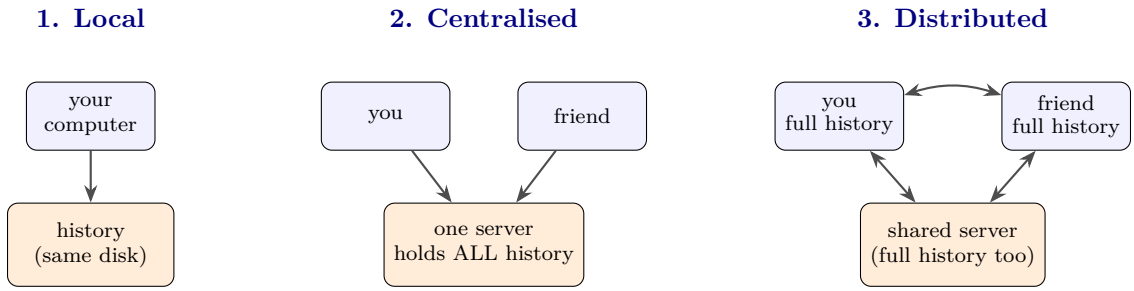


Figure 1: Where the history lives. In a centralised system the server is the only complete copy. In a distributed system (Git) *every* computer holds the complete history. Arrows show history being copied in either direction.

Local
The history is kept on one computer (for example RCS, 1982). Simple, but no real teamwork.

Centralised
One server holds the history; you download a working copy of the files and send changes back (for example CVS, Subversion). Teamwork is possible, but if the server is down or you have no internet, you cannot save history or look at old versions.

Distributed
Everyone has a full copy of the whole history (Git, Mercurial). You can work offline, save versions as often as you like, and copy history to or from any other computer, with or without a server. A shared server is still useful, but it is a convenience, not a requirement.

3 A short history of Git

This section is background. Read it for context; you will not be tested on dates.

History

Git was created in **April 2005** by Linus Torvalds, the creator of Linux. The Linux kernel team had been using a proprietary distributed tool called BitKeeper. When the free-use arrangement ended, the team needed a replacement. Torvalds wanted something fast, distributed, and safe against accidental corruption, and no existing free tool fit. He started writing Git on 3 April 2005. Within days Git could manage its own source code, and the kernel started using it that month. A few months later, Junio Hamano took over as maintainer, a role he still holds.

Year	Milestone
1972	SCCS: one of the first version-control tools (Bell Labs).
2000	Subversion (SVN) development starts (1.0 in 2004): a better centralised system, widely used for years.
2005	Git (Torvalds) and Mercurial appear within weeks of each other, both distributed.
2008	GitHub launches: a website for hosting Git repositories with tools for teamwork.
2018	Microsoft announces it will buy GitHub (the purchase completes the same year).
2020	New repositories on GitHub start on a branch called `main` instead of `master`; Git 2.28 lets you choose this name yourself (you do so in Homework 0).

Quick fact

- Git is free, open source, and works on Linux, macOS and Windows. The word "git" is British slang for an unpleasant person; Torvalds joked that he names his projects after himself.
- Every saved version in Git gets a unique ID: a 40-character code (by default) called a *hash*, such as `3f2a9c1...`. You will usually need only the first 7 characters (Lecture 3 shows where to see them).

4 Git is not GitHub

This is the most common confusion for beginners.

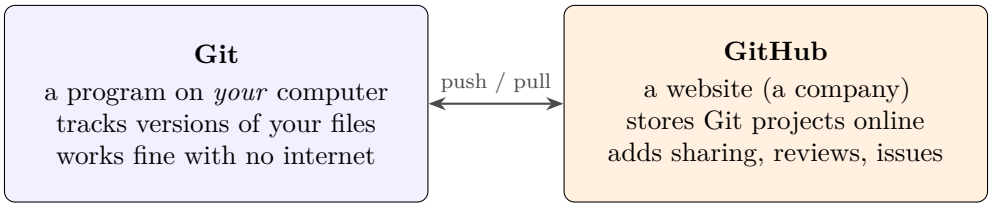


Figure 2: Git does the version tracking. GitHub is one place to keep a copy online and collaborate. Other sites (GitLab, Bitbucket, Codeberg) do a similar job.

An analogy: Git is like a very powerful "version history" feature, but for a whole folder, and it lives on your own computer. GitHub is like a shared cloud folder that stores that history online and lets friends comment on it. You can use Git without GitHub, but you cannot use GitHub's sharing without Git. In this course you learn Git first (Lectures 1–5), because GitHub's sharing features are built on top of it.

5 The order of the course

The order below rises in difficulty. **Each lecture reuses what you learned in the previous one.**

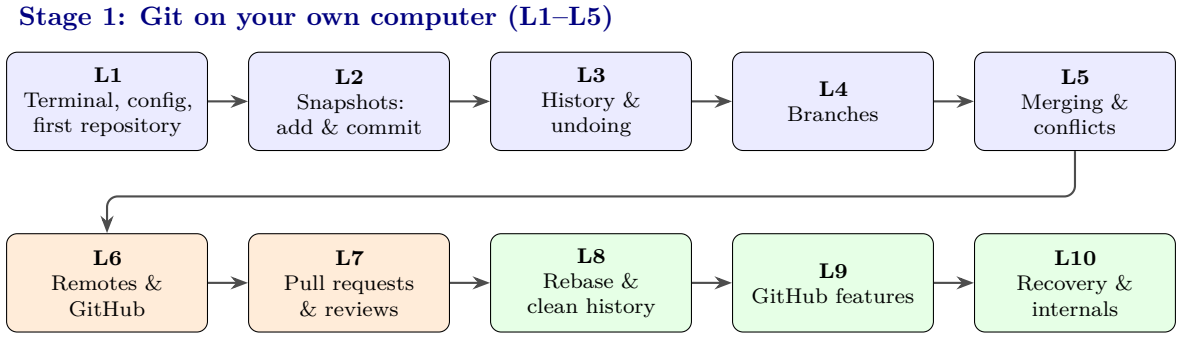


Figure 3: Course map. Blue (L1–5): Git on one computer. Orange (L6–7): sharing and teamwork. Green (L8–10): advanced tools. Follow the arrows.

#	Lecture	What you can do afterwards
1	First steps	Use the terminal; configure Git; turn a folder into a repository
2	Snapshots	Save versions with `add` and `commit`; ignore files
3	History & undoing	Read the history; compare versions; undo mistakes safely
4	Branches	Work on an idea separately without breaking the main line
5	Merging & conflicts	Combine branches; resolve a conflict calmly
6	Remotes & GitHub	Put your project on GitHub; `clone`, `push`, `pull`
7	Pull requests & reviews	Contribute to a friend's project; review each other's work
8	Rebase & cherry-pick	Tidy up history; cherry-pick; push rewritten history safely
9	GitHub features	Issues, Actions (automated checks), branch protection, releases
10	Recovery & internals	Rescue lost work (reflog, `reset`, `stash`, `bisect`); look inside `.git`

Why this order? GitHub's sharing is Git with a website on top, so we first learn Git alone (L1–5), where mistakes cannot affect anyone else. We first learn how a single copy of a project works. Rebase and recovery come last because they are most useful once you understand the simpler tools they replace or repair.

What you need. No programming experience. A computer, a text editor (any will do, even Notepad or TextEdit in plain-text mode), a terminal (a text window where you type commands instead of clicking), and a free GitHub account (needed from Lecture 6). Plan one to two hours per lecture.

6 How the homework works

Key idea

Type every command yourself. Reading about Git is not enough: your hands need to learn it too. Mistakes in a practice repository cost nothing.

Each lecture ends with a **Homework** box with tasks. You will do the work in the terminal. Most lectures come with three helpers:

1. **A setup script**, `codeHints/Lxx/setup_hwX.sh`, which builds a practice repository in the `practice/` folder, already in the starting state for the tasks.
2. **A checker**, `codeHints/Lxx/check_hwX.sh`, which looks at your repository and prints `PASS` or `FAIL` for each task (`SKIP` for an optional check that could not run).
3. **Solutions**, in two places: at the end of every lecture PDF, always starting on a new page, and as a separate PDF in `solutions/Lxx/`.

Some lectures also have a follow-along repository (`setup_demoN.sh`) and, from Lecture 6, a "friend" script that plays a teammate.

Here `Lxx` stands for a lecture folder such as `L02`, and `hwX` for its homework number. The course folder looks like this:

```
LearningGit/
  lectures/     the PDFs
  codeHints/    setup and check scripts for each lecture
  solutions/    solution PDFs
  practice/     your sandbox repositories (created by the setup scripts)
```

The workflow is: read the lecture, run the setup script, do the tasks, run the checker until everything says `PASS`, and *only then* open the solutions to compare your approach with the reference.

Always start in the course folder `LearningGit/`. (`cd` means "change directory"; Lecture 1 explains it.) Here is an *illustration* from Lecture 2; do not run it yet:

```
$ bash codeHints/L02/setup_hw2.sh     # build the practice repository
$ cd practice/hw2                     # go into it and do the tasks
$ cd ../..                            # back to the course folder
$ bash codeHints/L02/check_hw2.sh
PASS  setup: practice/hw2 is its own repository
FAIL  2.1/2.2/2.3 exactly three commits
```

If a check says `FAIL`, fix your repository and run the checker again.

In terminal listings, a line that starts with `$` is what *you* type (do not type the `$` itself). Other lines are what the computer prints. Text after `#` is a comment.

Note for Windows and macOS. The scripts run with the `bash` command. On Windows, use *Git Bash* (installed with Git for Windows). On macOS, `bash script.sh` works even though the default shell is `zsh`.

7 Homework 0

Homework 0: Get ready

You need a terminal and Git installed. A *terminal* is a window where you type commands. Open it as follows (Lecture 1 teaches the basics):

- **macOS:** open the app *Terminal* (Spotlight search). Typing `git --version` offers to install Git if it is missing.
- **Windows:** install Git from `https://git-scm.com/downloads`, then open *Git Bash* from the Start menu.
- **Linux:** open your terminal and install Git with your package manager, for example `sudo apt install git`.

0.1 Install

Install Git if needed (on Windows, open Git Bash only after installing), then run `git --version` in the terminal. It should print a version, such as `git version 2.43.0`. You need version 2.28 or newer (older versions do not support the setting in 0.3; if yours is older, update from `https://git-scm.com/downloads`).

0.2 Introduce yourself

Tell Git who you are. Every saved version records its author. Keep the quotation marks and replace the placeholder text with your own:

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
```

Use the same email you use (or will use) on GitHub.

0.3 Default branch name

Set the name of the first branch of new repositories to `main`. (A branch is a line of saved versions; you meet it in Lecture 4, so for now just copy the command.)

```
$ git config --global init.defaultBranch main
```

0.4 Short answer

In two or three sentences, explain the difference between Git and GitHub, as if to a friend who has never heard of either.

0.5 GitHub account

Create a free account at `https://github.com` (needed from Lecture 6).

0.6 Thinking

Name a time you wished you had version control.

Check yourself: run `bash codeHints/L00/check_hw0.sh` from the course folder (`LearningGit/`). It checks tasks 0.1–0.3. Tasks 0.4–0.6 you compare with the solutions.

8 Summary

- Version control records a project's history so you can compare, go back and collaborate. Git (Torvalds, 2005) is distributed: every copy holds the full history. Git is the tool on your computer; GitHub is a website that hosts Git projects.
- The course goes from Git on your own computer (L1–5) to sharing (L6–7) to advanced tools (L8–10). Further reading: *Pro Git*, free at `https://git-scm.com/book`, Chapter 1.

Solutions

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

Solutions to Homework 0

0.1. `git --version` prints something like `git version 2.43.0`. If you get "command not found", Git is not installed, or the terminal needs to be reopened after installing.

0.2. After running the two commands, check them with

```
$ git config --global user.name
Your Name
$ git config --global user.email
you@example.com
```

(Your own name and email will appear. `git config --global --list` shows all your settings.)

0.3. Check with `git config --global init.defaultBranch`. It should print `main`.

0.4. Model answer: "Git is a program on your computer that remembers every saved version of your files and lets you go back or combine work. GitHub is a website where you can store a copy of a Git project online to share it and work with others. You can use Git without GitHub, but not GitHub without Git."

0.5. Sign-up is checked by logging in at github.com. Nothing else to submit.

0.6. Any honest example is correct. Typical ones: a report or thesis where an old paragraph was deleted and needed again; two people editing different copies of the same file; "it worked yesterday, what did I change?"